

Expropriate (Hard Ver) 题解

题意

给定序列 $a_1, a_2, \dots, a_n$ 。一次操作选择一个元素 x ，将其变为

$$\text{inc}(x) = \min(x + 1, m),$$

并重新插入序列任意位置。同一元素可以被反复选择。求恰好执行 K 次操作后能得到的字典序最小序列。

题解

下文把数值相同但原位置不同的元素仍视为不同元素。

1. 保序骨架与自由袋

先固定每个元素的操作次数 c_i 。从未被操作的元素没有离开过序列，因此仍按原下标组成一个保序子序列

$$U = (a_i \mid c_i = 0).$$

每个被操作过的元素都可以在最后一次操作时重新插入任意位置。它们之间不再有顺序限制，只需记录最终值的多重集

$$B = \{\min(a_i + c_i, m) \mid c_i > 0\}.$$

于是，固定 (c_i) 后的最优答案就是 U 与升序多重集 B 的字典序最小稳定归并。归并时比较 U 的队首和 B 的最小值，取较小者；相等时先取 U 。两者当前输出值虽然相同，但先消耗 U 会更早解除保序骨架的队首约束，而袋中的同值元素仍可留到以后，因此不会更劣。

记该归并为 $\text{Merge}(U, B)$ 。此后所说的“删除一个活动元素”，实际含义都是“操作它一次并放入自由袋”，不是把它从最终答案中删去。

还会反复用到一个单调性：若两个局部答案满足

$$\text{Merge}(U_1, B_1) \leq \text{Merge}(U_2, B_2),$$

则对任意公共自由袋 C ，仍有

$$\text{Merge}(U_1, B_1 \uplus C) \leq \text{Merge}(U_2, B_2 \uplus C).$$

这是因为两边等价于再与同一个升序多重集 C 作稳定归并，而该操作保持字典序。它保证递推中只保存子问题的最优答案即可。

2. 把重复操作压到一个重元素

定义两个子问题：

$$\begin{aligned} D(A, k) &= \text{在 } A \text{ 中恰选 } k \text{ 个不同元素、每个操作一次时的最优答案,} \\ E(A, k) &= \min_{0 \leq s \leq k} D(A, s). \end{aligned}$$

E 除了返回最优字符串，还记录得到它所用的操作数 s 。若多个 s 得到同一个字符串，保留最大的 s 。

数学上 D/E 指归并后的字符串；递推与实现保存的是一个实现该字符串的三元组 (U, B, used) ，候选始终按 $\text{Merge}(U, B)$ 比较。

考虑一个存在重复操作的方案。先固定自由元素集合。对其升序最终值作交换，所有“第一次以外”的操作都可以集中到原值最大的自由元素 x 上：若某次额外增量仍落在其他元素上，把所有额外增量整体移给 x ，会保留更多较小的基准值，只把唯一变大的值推向有序袋末端。

若 x 还不是全局最大值，取一个尚未自由的全球最大值 M ，设 x 共承担 $c \geq 2$ 次操作。固定其他元素的操作次数，记 $\Phi(c_x, c_M)$ 为对应的最小稳定归并。逐位考察两条骨架首次经过 x, M 身份的位置，可以得到二元 uncrossing（解交叉交换）

$$\min(\Phi(0, c), \Phi(1, c-1)) \leq \Phi(c, 0).$$

左侧第一项把重身份直接交给 M ；第二项让 x 只操作一次，把其余次数交给 M 。若 $c = 2$ ，第二项恰把自由元素数增至 K ，落入 $D(A, K)$ ；若 $c > 2$ ，两项中的 M 都已是重元素。若 x 本身已经是全局最大值，只需在同值身份间交换；删去最早出现的最大值会把中间一段不大于 M 的元素提前，故不劣。

对任意含重复操作的方案，都存在一个不劣方案：要么 K 次操作作用于 K 个不同元素；要么原序列中最早出现的全球最大值 M 承担全部重复操作，其余元素至多操作一次。

直观上，额外的增量必然会使某个自由元素变大；把所有代价压到本来就最大的自由元素，能够让其余自由元素尽可能小，而这个重元素最终可以放到序列末尾。选择最早的全局最大值，则把一个尽可能靠前的最大障碍从保序骨架中移走。

令 p_M 为最早全局最大值的位置， A^- 为删除位置 p_M 后保持原顺序的序列。指定该元素为重元素，设其余元素共使用 s 次操作。重元素得到 $K - s \geq 2$ 次操作，最终值为

$$h_s = \min(M + K - s, m).$$

它不小于核心序列中的任何元素，故可以放在末尾。原问题只需比较

$$\text{Ans}(A, K) = \min(D(A, K), E(A^-, K-2) \parallel [h_s])$$

其中第二项仅在 $K \geq 2$ 时存在， s 是 $E(A^-, K-2)$ 实际采用的操作数。

这也解释了 E 的平局规则：核心字符串相同时， s 越大，留给重元素的操作越少， h_s 越小。

3. D/E 的统一递推

先忽略数据结构，只考察当前仍未决定是否操作的活动序列 X 。设

$$v = \min X, \quad w = v + 1,$$

并定义：

- p : 首个 v 之前的活动元素数；
- c : v 的活动出现次数；
- j : 首个原有 w 之前的活动元素数，不存在 w 时令 $j = |X|$ 。

D 把 k 当作必须用完的预算， E 把它当作操作数上界；除边界及同串平局外，两者使用完全相同的递推。进入 E 的每个状态时先令 $k \leftarrow \min(k, |X|)$ 。除 $v = m$ 外，递推中每次真正花费操作都会严格减小当前第一个未确定字符，所以“现在停止”被继续递推支配；当 $v = m$ 时字符串不再变化，直接按平局规则取最大的已用次数。

条件	保留的非支配决策	子问题预算
$p = 0$	批量固定前导 v	k
$0 < p \leq k$	操作首个 v 前的 p 个元素, 并固定该 v	$k - p$
$p > k, j > k$	批量操作最右侧的 $q = \min(c, k)$ 个 v	$k - q$
$p > k, j \leq k$	固定首个原有 w ; 仅当 $q = k - j + 1 \leq k$ 且 $c \geq q$ 时比较首次逃逸	$k - j / j - 1$ (若存在)

前导最小值。 若 $p = 0$, 前导 v 显然应直接保留。实现中一次固定最长的前导 v 段。对 E 可以整段固定; 对 D 至少要留下 k 个元素供以后操作, 因此至多固定 $|X| - k$ 个。

首个 v 可达。 若 $0 < p \leq k$, 可以操作其前面的 p 个元素, 使答案以 v 开头。任何以更大值开头的方案都被严格支配, 所以该选择是强制的。记这段前缀为 $P = X[0, p)$, 子问题答案为 (U', B') , 则本层返回

$$(v \parallel U', \text{inc}(P) \uplus B'),$$

并在首个 v 之后以预算 $k - p$ 递归。

v, w 均不可达。 若 $p > k$ 且 $j > k$, 保留原有 v 或 w 都无法得到当前首字符; 能得到的最小首值只能是操作一个 v 产生的 w 。若某方案操作了一个较左的 v 却保留较右的同值 v , 交换二者身份后, 自由袋不变而保序骨架不劣。因此操作 t 个最小值时总可规范为操作最右侧 t 个, 故本分支批量操作最后

$$q = \min(c, k)$$

个 v , 向自由袋加入 q 个 w , 再以预算 $k - q$ 递归。若还有预算, 说明所有 v 已被删空, 下一层的最小值会随之上升。

锚点与逃逸。 最关键的是 $p > k$ 且 $j \leq k$ 。由 $j \leq k < p$, 首个原有 w 出现在所有活动 v 之前, 因此之后操作的最右侧 v 全在锚点之后。此时存在两个非支配候选。

锚定候选 R 操作首个 w 前的 j 个元素, 保留这个 w :

$$R = (w \parallel U_R, \text{inc}(X[0, j]) \uplus B_R),$$

其子问题位于该 w 之后, 剩余预算为 $k - j$ 。

另一类方案先操作最右侧的若干个 v 。若操作数量 $t \leq k - j$, 这些操作完全发生在锚点之后, 已经由 R 的后缀子问题及其预算 $k - j$ 覆盖。第一次使锚点不可达的数量恰为

$$q = k - j + 1.$$

因此唯一需要另开的逃逸候选是: 先操作最后 q 个 v , 加入 q 个自由 w , 再以预算

$$k - q = j - 1$$

递归。所有 $t \geq q$ 的方案都可先做恰好 q 次, 再由该逃逸子问题覆盖。若 $q > k$ 或 $c < q$, 逃逸候选不存在; 否则比较锚定与逃逸所得的完整稳定归并结果。

综上, 每个状态至多真正分成两个子状态, 而且二分支只发生在锚点处。递推边界为:

- $X = \emptyset$: 返回空骨架、空自由袋, 已用次数为 0;
- $k = 0$: 整个活动序列保留;
- $v = m$: 继续操作不改变数值, 只更新 `used`;
- 精确模式且 $k = |X|$: 所有活动元素各操作一次, 整张频率表放入自由袋。

数学递推到此结束。余下只需解决三件实现问题：压缩活动序列、快速分裂频率，以及不实体化地比较两个候选。

4. 活动序列的闭包： (l, v, r)

上述递推不会产生任意删除集合。记 $\text{rk}_x(i)$ 为位置 i 在所有值为 x 的位置中的全局编号，从 0 开始。任意活动序列都可以写成

$$X(l, v, r) = (a_i \mid i \geq l, a_i > v \vee (a_i = v \wedge \text{rk}_v(i) < r)),$$

并保持原下标顺序。也就是说：

- 下标小于 l 的前缀已经处理；
- 小于 v 的值已经全部失活；
- 大于 v 的值全部有效；
- 只有当前最小值 v 可能删去了一段出现后缀，故仅保留其全局前 r 次出现。

这个状态族对全部转移封闭。固定首个最小值或锚点只会增大 l ；删除最右侧 q 个 v 只需令 $r \leftarrow r - q$ 。当当前 v 已经不存在时，从活动直方图取新的最小值 v' ；它此前没有被截断过，故令 r 等于 v' 在原序列中的总出现次数即可。

5. 位置版本与活动直方图

对每个 v 和 $0 \leq r \leq \text{cnt}(v)$ ，预处理一个按原下标建立的可持久化线段树版本

$$\text{ver}[v][r],$$

其中包含所有值大于 v 的位置，以及值等于 v 的前 r 个位置。按值从大到小扫描，先继承所有更大值，再依次加入 v 的出现位置即可。每个原位置只加入一次。

位置树结点维护所选元素数、最大值和按原下标顺序连接的哈希。对状态 (l, v, r) ，在 $\text{ver}[v][r]$ 中跳过位置小于 l 的元素，即可在 $O(\log n)$ 时间完成活动长度、活动秩与原位置互换、活动子串哈希，以及寻找第一个大于给定值的活动元素。

每个递归状态还维护一棵值域可持久化线段树 `hist`，精确记录当前活动元素的原值频率。它有两个用途：

1. 当前 v 被删空时，查询最小原值完成状态规范化；
2. 一段元素被统一操作一次时，直接将其频率树根挂入自由袋，读取时再惰性执行 $x \mapsto \text{inc}(x)$ 。

注意 `hist` 保存的是操作前的原值，已经算出最终值的重元素和常值段则直接存最终值。

6. small-side 分裂

递推经常需要在原位置 y 处分离活动前缀、锚点和后缀。若每次重新扫描前缀，复杂度会退化为平方。

为此，每个状态附带一个分界缓存

$$\text{Part} = (\text{cut}, H_{\text{pre}}, H_{\text{suf}}),$$

其中 H_{pre} 、 H_{suf} 分别是当前活动元素在 $[l, \text{cut})$ 与 $[\text{cut}, n)$ 中的频率树。当新分界从 `cut` 移到 y 时，只显式枚举能够重建两侧频率的较小一侧；另一侧由总频率与旧缓存作持久化加减得到。一次 `splitAt` 同时产生两个后继状态所需的缓存。

按被枚举的元素收费：每次收费后，它所在的相关区间大小至少减半，所以同一元素至多被枚举 $O(\log n)$ 次。所有分裂总共枚举 $O(n \log n)$ 个元素；取得每个活动值需 $O(\log n)$ ，并引发 $O(1)$ 次、每次 $O(\log(m+1))$ 的值域树修改。这是总复杂度中第二个对数的来源。

7. 惰性答案代数与候选比较

递推答案始终保存为三元组

$$(U, B, \text{used}),$$

而不立即生成长度为 n 的序列。

U 用按块隐式 Treap 表示。一个块要么是常值段，要么是某个位置树版本中的连续排名区间；结点维护长度、最大值与哈希。 B 用左偏树维护，一个堆块要么是已经确定最终值的常值段，要么是一棵读取时统一做 `inc` 的原值频率树。

比较或输出时，`MergedCursor` 惰性执行 $\text{Merge}(U, B)$ 。设自由袋当前最小值为 b ，则先整段输出 U 中不大于 b 的前缀；遇到第一个大于 b 的元素时，输出袋中全部等于 b 的元素。游标每次返回一整块，而不是单个元素。

候选比较使用模 2^{64} 自然溢出的单项式哈希

$$H(XY) = H(X) \text{BASE}^{|Y|} + H(Y) \pmod{2^{64}}.$$

两个游标同步产生块；当前等长前缀哈希不同时，在块内二分 LCP 并随机访问第一个不同值。只有最终胜出的答案才会被完整展开。

参考代码使用固定奇数底数和一个 `uint64_t` 哈希，属于面向非 Hack 型 ACM 环境的概率实现；不对刻意构造的碰撞给出形式保证。如需更强的抗构造性，可把底数运行时随机化或恢复双 64 位哈希。块 Treap 的复杂度按随机优先级模型分析，代码用伪随机数生成优先级。

正确性证明

引理 1 (自由化)。固定各元素操作次数后，所有可达结果恰为未操作保序骨架 U 与被操作元素最终值多重集 B 的交错； $\text{Merge}(U, B)$ 是其中字典序最小者。

证明。未操作元素从未离开序列，故相对顺序不变；每个被操作元素可以在最后一次操作时插入任意位置，故其位置任意。对固定的 U, B ，逐位取 U 队首与 B 最小值中的较小者是标准的字典序交换贪心。
□

引理 2 (操作次数正规形)。每个含重复操作的方案都被某个全异方案或某个“最早全局最大值为唯一重元素”的方案支配。

证明。固定自由集合时，把全部额外增量集中到原值最大的自由元素，得到的有序袋不劣。若它不是全局最大值，第 2 节的不等式表明：交换重身份与拆成两个自由身份至少有一个不劣。前者直接得到最大值重元素；后者在重次数大于 2 时仍得到最大值重元素，在重次数等于 2 时落入 $D(A, K)$ 。相等最大值中删除更早者不劣，饱和只会令增量失效，不改变交换方向。□

引理 3 (递推完备性)。第 3 节的四种情况保留了 D/E 的全部非支配候选。

证明。若首个 v 可达，以 v 开头严格优于任何更大的首值，故必须固定它。若不可达，最小可能首值为 $w = v + 1$ ；一个 w 要么来自最右侧的某个 v ，要么来自可达的首个原有 w 。原有 w 不可达时，只能连续制造 w ，直至预算用尽或 v 被删空。原有 w 可达时，先制造至多 $k - j$ 个 w 的方案仍被锚定递归覆盖； $q = k - j + 1$ 是第一次逃离锚定分支的状态，更多制造操作又由该逃逸子问题覆盖。因此最多只需比较锚定与逃逸。□

引理 4 (状态闭包)。任意递归子问题的活动序列都能由某个 (l, v, r) 表示，递推始终维护其中一个规范状态；位置版本与活动直方图给出的查询结果与该序列一致。

证明。初态显然满足定义。固定元素只删除一个原前缀，修改 l ；制造自由 w 时始终删除最右侧的 v ，修改 r 。只有当前最小值会被这样截断，所以它消失后，新的最小值仍保留全部出现。归纳即得。□

定理。 算法输出恰好执行 K 次操作后可得的字典序最小序列。

证明。 引理 1 给出固定操作次数分配时的最优结果；引理 2 将所有分配划分为 $D(A, K)$ 与重元素正规形族。对 D/E 按活动长度和预算归纳，引理 3 保证至少一个最优方案位于递推保留的候选中，引理 4 保证实现维护的正是对应子问题。两类顶层候选取较小者，即为全局最优。□

复杂度

位置版本共加入 $O(n)$ 个点，使用 $O(n \log n)$ 个结点。唯一产生二分支的锚点状态满足

$$(k - j) + (j - 1) = k - 1;$$

其余转移则降低预算或固定至少一个活动元素，故递推状态总数为 $O(n)$ 。small-side 分裂总共枚举 $O(n \log n)$ 个元素，每次作 $O(\log(m + 1))$ 的持久化频率修改。答案游标访问的块总数可同样向状态分裂收费为 $O(n \log n)$ ，每次块查询或 LCP 至多再带一个 $O(\log n)$ 。程序至多建立两个 `Engine`，只增加常数因子。

- 时间复杂度：期望摊还 $O(n \log^2 n)$ 。
- 空间复杂度： $O(n \log^2 n)$ 。

这里使用了题目条件 $m \leq 2n$ ；若保留一般值域，初始化还需 $O(m)$ ，并应把值域对数单独写作 $\log(m + 1)$ 。正确性按通常的单 64 位哈希模型忽略碰撞；复杂度中的“期望”来自块 Treap 的随机优先级模型。

参考代码

```
/*
D(A,k): 恰好选 k 个不同元素各操作一次；E(A,k)=min_{s<=k}D(A,s)。
含重复操作的方案要么被 D(A,k) 支配，要么可令最早全局最大值承担全部重复操作。
D/E 的状态 (l,v,r) 表示只看位置 >=l，值 >v 全保留，值 v 仅保留前 r 次出现。
递推结果记为 (U,B)：未操作子序列 U 与已操作元素有序袋 B，答案是二者的稳定归并。
单 64 位自然溢出哈希比较候选；在 m=O(n) 下，期望摊还时间、空间均为 O(n log^2 n)。
*/
#include <bits/stdc++.h>
using namespace std;

using u64 = uint64_t;

struct Info {
    int len = 0, mx = -1;
    u64 h = 0;
};

constexpr u64 BASE = 11995408973635179863ULL;
vector<u64> pw, rep;

void initHash(int n) {
    pw.assign(n + 1, 1);
    rep.assign(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        pw[i] = pw[i - 1] * BASE;
        rep[i] = rep[i - 1] * BASE + 1;
    }
}

Info join(const Info &a, const Info &b) {
```

```

        return {a.len + b.len, max(a.mx, b.mx), a.h * pw[b.len] + b.h};
    }

    Info runInfo(int value, int len) {
        return len ? Info{len, value, (u64(value) + 1) * rep[len]} : Info{};
    }

    struct PostTree {
        struct Node {
            int l = 0, r = 0;
            Info a{};
        };

        int n;
        vector<Node> tr;

        explicit PostTree(int n_) : n(n_) {
            tr.reserve(max(2, n * 20));
            tr.push_back({});
        }

        int add(int old, int q, int value, int lo, int hi) {
            int z = int(tr.size());
            tr.push_back(tr[old]);

            if (hi - lo == 1) {
                tr[z].a = runInfo(value, 1);
                return z;
            }

            int md = (lo + hi) >> 1;
            if (q < md) {
                tr[z].l = add(tr[old].l, q, value, lo, md);
            } else {
                tr[z].r = add(tr[old].r, q, value, md, hi);
            }

            tr[z].a = join(tr[tr[z].l].a, tr[tr[z].r].a);
            return z;
        }

        int add(int old, int q, int value) { return add(old, q, value, 0, n); }

        int countPosPrefix(int z, int q, int lo, int hi) const {
            if (!z || q <= lo) return 0;
            if (hi <= q) return tr[z].a.len;

            int md = (lo + hi) >> 1;
            return countPosPrefix(tr[z].l, q, lo, md) + countPosPrefix(tr[z].r, q,
md, hi);
        }

        int countPosPrefix(int z, int q) const { return countPosPrefix(z, q, 0, n); }
    }

```

```

Info queryRank(int z, int ql, int qr) const {
    if (!z || ql >= qr) return {};

    const int sz = tr[z].a.len;
    if (ql == 0 && qr == sz) return tr[z].a;

    int ls = tr[tr[z].l].a.len;
    Info ans;

    if (ql < ls) {
        ans = queryRank(tr[z].l, ql, min(qr, ls));
    }
    if (qr > ls) {
        ans = join(ans, queryRank(tr[z].r, max(0, ql - ls), qr - ls));
    }

    return ans;
}

int valueAtRank(int z, int k) const {
    while (tr[z].a.len > 1) {
        int ls = tr[tr[z].l].a.len;
        if (k < ls) {
            z = tr[z].l;
        } else {
            k -= ls;
            z = tr[z].r;
        }
    }
    return tr[z].a.mx;
}

int positionAtRank(int z, int k) const {
    int lo = 0, hi = n;
    while (hi - lo > 1) {
        int md = (lo + hi) >> 1;
        int ls = tr[tr[z].l].a.len;
        if (k < ls)
            z = tr[z].l, hi = md;
        else
            k -= ls, z = tr[z].r, lo = md;
    }
    return lo;
}

int firstGreaterRank(int z, int ql, int qr, int value) const {
    if (!z || ql >= qr || tr[z].a.mx <= value) return -1;

    int sz = tr[z].a.len;
    if (sz == 1) return 0;

    int ls = tr[tr[z].l].a.len;
    if (ql < ls) {
        int got = firstGreaterRank(tr[z].l, ql, min(qr, ls), value);
        if (got != -1) return got;
    }

```

```

    }

    if (qr > ls) {
        int got = firstGreaterRank(tr[z].r, max(0, ql - ls), qr - ls,
value);
        if (got != -1) return ls + got;
    }

    return -1;
}
};

struct CntTree {
    struct Node {
        int l = 0, r = 0, cnt = 0;
    };

    int sigma;
    vector<Node> tr;

    explicit CntTree(int sigma_) : sigma(sigma_) { tr.push_back({}); }

    int update(int old, int q, int delta, int lo, int hi) {
        int z = int(tr.size());
        tr.push_back(tr[old]);
        tr[z].cnt += delta;

        if (hi - lo == 1) return z;

        int md = (lo + hi) >> 1;
        if (q < md) {
            tr[z].l = update(tr[old].l, q, delta, lo, md);
        } else {
            tr[z].r = update(tr[old].r, q, delta, md, hi);
        }

        return z;
    }

    int update(int old, int q, int delta) { return update(old, q, delta, 0,
sigma); }

    int countLess(int z, int q, int lo, int hi) const {
        if (!z || q <= lo) return 0;
        if (hi <= q) return tr[z].cnt;

        int md = (lo + hi) >> 1;
        return countLess(tr[z].l, q, lo, md) + countLess(tr[z].r, q, md, hi);
    }

    int countLess(int z, int q) const { return countLess(z, q, 0, sigma); }

    int rawValueAtRank(int z, int k) const {
        int lo = 0, hi = sigma;

```

```

        while (hi - lo > 1) {
            int md = (lo + hi) >> 1;
            int ls = tr[tr[z].l].cnt;

            if (k < ls) {
                z = tr[z].l;
                hi = md;
            } else {
                k -= ls;
                z = tr[z].r;
                lo = md;
            }
        }

        return lo;
    }

    int valueAtRank(int z, int k) const { return min(rawValueAtRank(z, k) + 1,
sigma - 1); }

    int countMappedLess(int z, int value) const {
        if (value >= sigma) return tr[z].cnt;
        return countLess(z, value - 1);
    }
};

struct BagHeap {
    struct Piece {
        int type = 0;
        int x = 0, start = 0, len = 0;
    };

    struct Node {
        int l = 0, r = 0, dist = 1, key = 0;
        Piece piece{};
    };

    const CntTree &pst;
    vector<Node> tr;

    explicit BagHeap(const CntTree &pst_) : pst(pst_) {
        tr.push_back({});
        tr[0].dist = 0;
    }

    int pieceMin(const Piece &p) const { return p.type == 0 ? p.x :
pst.valueAtRank(p.x, p.start); }

    int meld(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (tr[a].key > tr[b].key) swap(a, b);

        tr[a].r = meld(tr[a].r, b);
        if (tr[tr[a].l].dist < tr[tr[a].r].dist) {
            swap(tr[a].l, tr[a].r);

```

```

    }
    tr[a].dist = tr[tr[a].r].dist + 1;

    return a;
}

int addPiece(int root, Piece p) {
    if (!p.len) return root;

    int z = int(tr.size());
    tr.push_back({0, 0, 1, pieceMin(p), p});
    return meld(root, z);
}

int addRun(int root, int value, int count) { return addPiece(root, {0,
value, 0, count}); }

int addHist(int root, int histRoot) {
    int len = histRoot ? pst.tr[histRoot].cnt : 0;
    return addPiece(root, {1, histRoot, 0, len});
}
};

struct SeqTree {
    struct Node {
        int l = 0, r = 0;
        unsigned pri = 0;
        int type = 0;
        int x = 0, y = 0;
        Info one{}, all{};
    };

    const PostTree &pst;
    vector<Node> tr;
    u64 rngState = 0x243f6a8885a308d3ULL;

    explicit SeqTree(const PostTree &pst_) : pst(pst_) { tr.push_back({}); }

    unsigned rnd() {
        rngState ^= rngState << 7;
        rngState ^= rngState >> 9;
        return unsigned(rngState);
    }

    int alloc() {
        tr.push_back({});
        return int(tr.size()) - 1;
    }

    Info all(int z) const { return z ? tr[z].all : Info{}; }

    void pull(int z) { tr[z].all = join(join(all(tr[z].l), tr[z].one),
all(tr[z].r)); }

    int makeRun(int value, int count) {

```

```

    if (!count) return 0;

    int z = alloc();
    tr[z].pri = rnd();
    tr[z].type = 0;
    tr[z].x = value;
    tr[z].y = 0;
    tr[z].one = runInfo(value, count);
    tr[z].all = tr[z].one;

    return z;
}

int makeslice(int root, int start, int len) {
    if (!len) return 0;

    int z = alloc();
    tr[z].pri = rnd();
    tr[z].type = 1;
    tr[z].x = root;
    tr[z].y = start;
    tr[z].one = pst.queryRank(root, start, start + len);
    tr[z].all = tr[z].one;

    return z;
}

int merge(int a, int b) {
    if (!a || !b) return a ? a : b;

    if (tr[a].pri > tr[b].pri) {
        tr[a].r = merge(tr[a].r, b);
        pull(a);
        return a;
    }

    tr[b].l = merge(a, tr[b].l);
    pull(b);
    return b;
}

int firstGreaterFrom(int z, int skip, int value) const {
    if (!z || skip >= tr[z].all.len || tr[z].all.mx <= value) {
        return -1;
    }

    int ls = all(tr[z].l).len;
    if (skip < ls) {
        int got = firstGreaterFrom(tr[z].l, skip, value);
        if (got != -1) return got;
    }

    int chunkSkip = max(0, skip - ls);
    if (chunkSkip < tr[z].one.len && tr[z].one.mx > value) {
        int got = -1;

```

```

        if (tr[z].type == 0) {
            got = tr[z].x > value ? chunkSkip : -1;
        } else {
            int q = pst.firstGreaterRank(tr[z].x, tr[z].y + chunkSkip,
tr[z].y + tr[z].one.len,
                                value);
            if (q != -1) got = q - tr[z].y;
        }

        if (got != -1) return ls + got;
    }

    int rightSkip = max(0, skip - ls - tr[z].one.len);
    int got = firstGreaterFrom(tr[z].r, rightSkip, value);

    return got == -1 ? -1 : ls + tr[z].one.len + got;
}

int prependRun(int root, int value, int count = 1) {
    return merge(makeRun(value, count), root);
}

Info prefix(int z, int k) const {
    if (!z || k <= 0) return {};
    if (k >= tr[z].all.len) return tr[z].all;

    int ls = all(tr[z].l).len;
    if (k <= ls) return prefix(tr[z].l, k);

    Info ans = all(tr[z].l);
    int take = min(k - ls, tr[z].one.len);
    Info mid;

    if (take == tr[z].one.len) {
        mid = tr[z].one;
    } else if (tr[z].type == 0) {
        mid = runInfo(tr[z].x, take);
    } else {
        mid = pst.queryRank(tr[z].x, tr[z].y, tr[z].y + take);
    }

    ans = join(ans, mid);

    if (k > ls + tr[z].one.len) {
        ans = join(ans, prefix(tr[z].r, k - ls - tr[z].one.len));
    }

    return ans;
}

u64 rangeHash(int z, int l, int r) const {
    if (l >= r) return {};

    u64 a = prefix(z, l).h;

```

```

        u64 b = prefix(z, r).h;
        int len = r - 1;
        return b - a * pw[len];
    }

    int at(int z, int k) const {
        int ls = all(tr[z].l).len;
        if (k < ls) return at(tr[z].l, k);

        k -= ls;
        if (k < tr[z].one.len) {
            if (tr[z].type == 0) return tr[z].x;
            return pst.valueAtRank(tr[z].x, tr[z].y + k);
        }

        return at(tr[z].r, k - tr[z].one.len);
    }
};

struct Engine {
    struct State {
        int l = 0, v = 0, r = 0;
    };

    struct Part {
        int cut = 0, pre = 0, suf = 0;
    };

    struct Split {
        int prefix = 0, suffix = 0;
        Part keepPart{}, movePart{};
    };

    struct Answer {
        int seqRoot = 0, bag = 0, used = 0;
    };

    int n, m;
    vector<int> a;
    vector<vector<int>> occ, ver;
    PostTree pos;
    CntTree bags;
    SeqTree seq;
    BagHeap heap;

    Engine(vector<int> a_, int m_)
        : n(int(a_.size())), m(m_), a(move(a_)), occ(m + 1), ver(m + 1), pos(n),
bags(m + 1),
        seq(pos), heap(bags) {
        for (int i = 0; i < n; ++i) {
            occ[a[i]].push_back(i);
        }

        int higher = 0;
        for (int v = m; v >= 0; --v) {

```

```

        ver[v].reserve(occ[v].size() + 1);
        ver[v].push_back(higher);

        int cur = higher;
        for (int p : occ[v]) {
            cur = pos.add(cur, p, v);
            ver[v].push_back(cur);
        }
        higher = cur;
    }
}

State normalize(State s, int hist) const {
    if (!bags.tr[hist].cnt) return {n, m + 1, 0};
    int nv = bags.rawValueAtRank(hist, 0);
    if (nv != s.v) s.v = nv, s.r = int(occ[nv].size());
    return s;
}

int versionRoot(const State &s) const { return s.v <= m ? ver[s.v][s.r] : 0;
}

int startRank(const State &s) const {
    return s.l >= n ? 0 : pos.countPosPrefix(versionRoot(s), s.l);
}

int activeLen(const State &s) const {
    if (s.l >= n || s.v > m) return 0;

    int rt = versionRoot(s);
    return pos.tr[rt].a.len - pos.countPosPrefix(rt, s.l);
}

int activeRankBefore(const State &s, int originalPos) const {
    int rt = versionRoot(s);
    return pos.countPosPrefix(rt, originalPos) - pos.countPosPrefix(rt,
s.l);
}

int originalPosAtActiveRank(const State &s, int rank) const {
    return pos.positionAtRank(versionRoot(s), startRank(s) + rank);
}

int makeActiveword(const State &s) {
    int rt = versionRoot(s);
    int st = startRank(s);
    int len = pos.tr[rt].a.len - st;
    return seq.makeSlice(rt, st, len);
}

int histAdd(int root, int value, int delta) {
    return delta ? bags.update(root, value, delta) : root;
}

vector<int> activeValues(const State &s, int l, int r) const {

```

```

vector<int> z;
if (l >= r) return z;

int rt = versionRoot(s);
int ql = pos.countPosPrefix(rt, l);
int qr = pos.countPosPrefix(rt, r);

z.reserve(qr - ql);
for (int i = ql; i < qr; ++i) {
    z.push_back(pos.valueAtRank(rt, i));
}

return z;
}

int addValues(int root, const vector<int> &z, int delta) {
    for (int x : z) {
        root = histAdd(root, x, delta);
    }
    return root;
}

int buildValues(const vector<int> &z) { return addValues(0, z, 1); }

Split splitAt(const State &s, int hist, const Part &c, int y) {
    const int val = a[y];
    Split out;

    if (y < c.cut) {
        int p = activeRankBefore(s, y);
        int x = activeRankBefore(s, c.cut) - p - 1;

        if (p <= x) {
            vector<int> v = activeValues(s, s.l, y);
            out.prefix = buildValues(v);

            int noPrefix = addValues(hist, v, -1);
            out.suffix = histAdd(noPrefix, val, -1);

            int middle = histAdd(addValues(c.pre, v, -1), val, -1);

            out.keepPart = {c.cut, middle, c.suf};
            out.movePart = {y, out.prefix, noPrefix};
        } else {
            vector<int> v = activeValues(s, y + 1, c.cut);
            int middle = buildValues(v);

            out.prefix = histAdd(addValues(c.pre, v, -1), val, -1);
            out.suffix = addValues(c.suf, v, 1);

            int anchorAndSuffix = histAdd(out.suffix, val, 1);

            out.keepPart = {c.cut, middle, c.suf};
            out.movePart = {y, out.prefix, anchorAndSuffix};
        }
    }
}

```

```

    } else if (y == c.cut) {
        out.prefix = c.pre;
        out.suffix = histAdd(c.suf, val, -1);
        out.keepPart = {y + 1, 0, out.suffix};
        out.movePart = {y, out.prefix, c.suf};
    } else {
        int d = activeRankBefore(s, y) - activeRankBefore(s, c.cut);
        int after = activeLen(s) - activeRankBefore(s, y) - 1;

        if (d <= after) {
            vector<int> v = activeValues(s, c.cut, y);
            out.prefix = addValues(c.pre, v, 1);

            int noMiddle = addValues(c.suf, v, -1);
            out.suffix = histAdd(noMiddle, val, -1);

            out.keepPart = {y + 1, 0, out.suffix};
            out.movePart = {y, out.prefix, noMiddle};
        } else {
            vector<int> v = activeValues(s, y + 1, n);
            out.suffix = buildValues(v);
            out.prefix = histAdd(addValues(hist, v, -1), val, -1);

            int anchorAndSuffix = histAdd(out.suffix, val, 1);

            out.keepPart = {y + 1, 0, out.suffix};
            out.movePart = {y, out.prefix, anchorAndSuffix};
        }
    }

    return out;
}

pair<int, Part> deleteLastMin(const State &s, int hist, Part c, int q) {
    int from = s.r - q;
    int cut = int(lower_bound(occ[s.v].begin(), occ[s.v].end(), c.cut) -
occ[s.v].begin());

    int qPre = clamp(min(s.r, cut) - from, 0, q);

    hist = histAdd(hist, s.v, -q);
    c.pre = histAdd(c.pre, s.v, -qPre);
    c.suf = histAdd(c.suf, s.v, -(q - qPre));

    return {hist, c};
}

pair<int, Part> removeLeading(const State &s, int hist, Part c, int take,
int newL) {
    int firstRank = int(lower_bound(occ[s.v].begin(), occ[s.v].end(), s.l) -
occ[s.v].begin());
    int cut = int(lower_bound(occ[s.v].begin(), occ[s.v].end(), c.cut) -
occ[s.v].begin());

    int qPre = clamp(cut - firstRank, 0, take);

```

```

hist = histAdd(hist, s.v, -take);
c.pre = histAdd(c.pre, s.v, -qPre);
c.suf = histAdd(c.suf, s.v, -(take - qPre));

if (newL > c.cut) {
    c = {newL, 0, hist};
}

return {hist, c};
}

struct Chunk {
    int type = 0, x = 0, start = 0, len = 0;
};

struct BagCursor {
    struct Entry {
        int key = 0, heapNode = 0;
        BagHeap::Piece piece{};
    };

    struct Greater {
        bool operator()(const Entry &a, const Entry &b) const { return a.key
> b.key; }
    };

    const Engine &e;
    priority_queue<Entry, vector<Entry>, Greater> pq;

    BagCursor(const Engine &e_, int root) : e(e_) {
        if (root) pushHeap(root);
    }

    void pushHeap(int z) {
        if (z) pq.push({e.heap.tr[z].key, z, {}});
    }

    void pushPiece(BagHeap::Piece p) {
        if (p.len) pq.push({e.heap.piecesMin(p), 0, p});
    }

    void expandOne() {
        Entry q = pq.top();
        pq.pop();

        const auto &z = e.heap.tr[q.heapNode];
        pushHeap(z.l);
        pushHeap(z.r);
        pushPiece(z.piece);
    }

    int peek() {
        while (!pq.empty() && pq.top().heapNode) expandOne();
    }
}

```

```

        return pq.empty() ? INT_MAX : pq.top().key;
    }

    pair<int, int> consumeRun() {
        int value = peek();
        int total = 0;

        while (!pq.empty() && pq.top().key == value) {
            if (pq.top().heapNode) {
                expandOne();
                continue;
            }

            Entry q = pq.top();
            pq.pop();
            auto p = q.piece;
            int take;

            if (p.type == 0) {
                take = p.len;
            } else {
                int cut = e.bags.countMappedLess(p.x, value + 1);
                take = min(p.start + p.len, cut) - p.start;
            }

            total += take;
            p.start += take;
            p.len -= take;
            pushPiece(p);
        }

        return {value, total};
    }
};

struct MergedCursor {
    const Engine &e;
    int seqRoot = 0, pos = 0, len = 0;
    BagCursor bag;

    MergedCursor(const Engine &e_, const Answer &r)
        : e(e_), seqRoot(r.seqRoot), len(e_.seq.all(r.seqRoot).len), bag(e_,
r.bag) {}

    bool next(Chunk &out) {
        int b = bag.peek();

        if (pos < len) {
            if (b == INT_MAX) {
                out = {0, seqRoot, pos, len - pos};
                pos = len;
                return true;
            }

            int p = e.seq.firstGreaterFrom(seqRoot, pos, b);

```

```

        if (p == -1) {
            out = {0, seqRoot, pos, len - pos};
            pos = len;
            return true;
        }

        if (p > pos) {
            out = {0, seqRoot, pos, p - pos};
            pos = p;
            return true;
        }
    }

    if (b == INT_MAX) return false;

    auto [value, count] = bag.consumeRun();
    out = {1, value, 0, count};
    return true;
}

};

u64 chunkHash(const Chunk &c, int off, int len) const {
    return c.type == 0 ? seq.rangeHash(c.x, c.start + off, c.start + off +
len)
                        : runInfo(c.x, len).h;
}

int chunkAt(const Chunk &c, int off) const {
    return c.type == 0 ? seq.at(c.x, c.start + off) : c.x;
}

static int compareResults(const Engine &a, const Answer &ra, const Engine
&b,
                        const Answer &rb) {
    MergedCursor ia(a, ra), ib(b, rb);
    Chunk ca, cb;
    int pa = 0, pb = 0;
    bool ha = ia.next(ca);
    bool hb = ib.next(cb);

    while (ha && hb) {
        int take = min(ca.len - pa, cb.len - pb);

        u64 xa = a.chunkHash(ca, pa, take);
        u64 xb = b.chunkHash(cb, pb, take);

        if (xa != xb) {
            int lo = 0, hi = take;

            while (lo < hi) {
                int md = (lo + hi + 1) >> 1;
                if (a.chunkHash(ca, pa, md) == b.chunkHash(cb, pb, md)) {
                    lo = md;
                } else {

```

```

        hi = md - 1;
    }
}

int va = a.chunkAt(ca, pa + 1o);
int vb = b.chunkAt(cb, pb + 1o);
return va < vb ? -1 : 1;
}

pa += take;
pb += take;

if (pa == ca.len) {
    pa = 0;
    ha = ia.next(ca);
}
if (pb == cb.len) {
    pb = 0;
    hb = ib.next(cb);
}

if (ha != hb) return ha ? 1 : -1;
return 0;
}

```

```

vector<int> materialize(const Answer &r) const {
    MergedCursor it(*this, r);
    Chunk c;
    vector<int> out;
    out.reserve(n);

    while (it.next(c)) {
        if (c.type == 1) {
            out.insert(out.end(), c.len, c.x);
        } else {
            for (int i = 0; i < c.len; ++i) {
                out.push_back(seq.at(c.x, c.start + i));
            }
        }
    }

    return out;
}

```

```

Answer solveIter(State initial, int initialK, bool atMost, int initialHist,
Part initialPart) {
    enum Stage { ENTER, PREFIX, KEEP, MOVE, ANCHOR, CHOOSE };
    struct Frame {
        State s{};
        Part part{}, escapePart{};
        int k = 0, hist = 0;
        Stage stage = ENTER;
        int value = 0, count = 0, bag = 0;
        int usedAdd = 0, q = 0, j = 0;
    };
}

```

```

        Answer candidate{};
    };

    auto frame = [](State s, int k, int hist, Part part) {
        Frame f;
        f.s = s;
        f.k = k;
        f.hist = hist;
        f.part = part;
        return f;
    };

    vector<Frame> st;
    st.reserve(2 * n + 8);
    st.push_back(frame(initial, initialK, initialHist, initialPart));

    Answer last;

    while (!st.empty()) {
        Frame &f = st.back();

        if (PREFIX <= f.stage && f.stage <= MOVE) {
            if (f.stage == KEEP) last.bag = heap.addHist(last.bag, f.bag);
            if (f.stage == MOVE)
                last.bag = heap.addRun(last.bag, f.value, f.count);
            else
                last.seqRoot = seq.prependRun(last.seqRoot, f.value,
f.count);

            last.used += f.usedAdd;
            st.pop_back();
            continue;
        }

        if (f.stage == ANCHOR) {
            Answer r = last;
            r.bag = heap.addHist(r.bag, f.bag);
            r.seqRoot = seq.prependRun(r.seqRoot, f.value);
            r.used += f.usedAdd;
            r.candidate = r;

            auto [moveHist, escapePart] = deleteLastMin(f.s, f.hist,
f.escapePart, f.q);

            State ms{f.s.l, f.s.v, f.s.r - f.q};
            int childK = f.j - 1;

            f.stage = CHOOSE;
            st.push_back(frame(ms, childK, moveHist, escapePart));
            continue;
        }

        if (f.stage == CHOOSE) {
            Answer moved = last;
            moved.bag = heap.addRun(moved.bag, f.value, f.q);
            moved.used += f.q;

```

```

        Answer anchored = f.candidate;
        int cmp = compareResults(*this, anchored, *this, moved);

        if (cmp < 0 || (cmp == 0 && (!atMost || anchored.used >=
moved.used))) {
            last = anchored;
        } else {
            last = moved;
        }

        st.pop_back();
        continue;
    }

    f.s = normalize(f.s, f.hist);
    int len = activeLen(f.s);

    if (atMost) {
        f.k = min(f.k, len);
    }

    if (!len) {
        last = {};
        st.pop_back();
        continue;
    }

    if (f.k == 0) {
        last = {makeActiveword(f.s), 0, 0};
        st.pop_back();
        continue;
    }

    if (f.s.v == m) {
        last = {makeActiveword(f.s), 0, f.k};
        st.pop_back();
        continue;
    }

    if (!atMost && f.k == len) {
        last = {0, heap.addHist(0, f.hist), f.k};
        st.pop_back();
        continue;
    }

    int lb =
        int(lower_bound(occ[f.s.v].begin(), occ[f.s.v].end(), f.s.l) -
occ[f.s.v].begin());
    int firstv = occ[f.s.v][lb];
    int p = activeRankBefore(f.s, firstv);

    if (p == 0) {
        int rt = versionRoot(f.s);
        int sr = startRank(f.s);

```

```

    int fg = pos.firstGreaterRank(rt, sr, sr + len, f.s.v);

    int take = (fg == -1 ? len : fg - sr);
    if (!atMost) take = min(take, len - f.k);
    int lastPos = originalPosAtActiveRank(f.s, take - 1);

    auto [nextHist, nextPart] = removeLeading(f.s, f.hist, f.part,
take, lastPos + 1);
    State ch{lastPos + 1, f.s.v, f.s.r};

    f.stage = PREFIX;
    f.value = f.s.v;
    f.count = take;
    st.push_back(frame(ch, f.k, nextHist, nextPart));
    continue;
}

if (p <= f.k) {
    Split sp = splitAt(f.s, f.hist, f.part, firstV);

    State ch{firstV + 1, f.s.v, f.s.r};
    int childK = f.k - p;

    f.stage = KEEP;
    f.value = f.s.v;
    f.count = 1;
    f.bag = sp.prefix;
    f.usedAdd = p;

    st.push_back(frame(ch, childK, sp.suffix, sp.keepPart));
    continue;
}

int w = f.s.v + 1;
int cntV = f.s.r - 1b;

auto itw = lower_bound(occ[w].begin(), occ[w].end(), f.s.l);

int firstW = (itw == occ[w].end() ? n : *itw);
int j = (firstW == n ? len : activeRankBefore(f.s, firstW));

if (j > f.k) {
    int q = min(cntV, f.k);

    auto [nextHist, nextPart] = deleteLastMin(f.s, f.hist, f.part,
q);

    State ch{f.s.l, f.s.v, f.s.r - q};
    int childK = f.k - q;

    f.stage = MOVE;
    f.value = w;
    f.count = q;
    f.usedAdd = q;

```

```

        st.push_back(frame(ch, childk, nextHist, nextPart));
        continue;
    }

    Split sp = splitAt(f.s, f.hist, f.part, firstw);

    int q = f.k - j + 1;
    State rs{firstw + 1, f.s.v, f.s.r};
    int childk = f.k - j;

    f.value = w;
    f.bag = sp.prefix;
    f.usedAdd = j;

    if (q > f.k || cntV < q) {
        f.stage = KEEP;
        f.count = 1;
    } else {
        f.stage = ANCHOR;
        f.q = q;
        f.j = j;
        f.escapePart = sp.movePart;
    }

    st.push_back(frame(rs, childk, sp.suffix, sp.keepPart));
}

return last;
}

Answer solve(int k, bool atMost) {
    int v = *min_element(a.begin(), a.end());
    int hist = 0;
    for (int x : a) hist = histAdd(hist, x, 1);
    return solveIter({0, v, int(occ[v].size())}, k, atMost, hist, {0, 0,
hist});
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, k, m;
        cin >> n >> k >> m;
        vector<int> a(n);
        for (int &x : a) cin >> x;

        vector<int> ans;
        if (k == 0 || *min_element(a.begin(), a.end()) == m) {
            ans = a;
        } else if (n == 1) {

```

```

        ans = {min(m, a[0] + k)};
    } else {
        initHash(n);
        Engine ex(a, m);
        auto best = ex.solve(k, false);

        if (k < 2) {
            ans = ex.materialize(best);
        } else {
            auto it = max_element(a.begin(), a.end());
            int mx = *it, at = int(it - a.begin());
            vector<int> b = a;
            b.erase(b.begin() + at);

            Engine ep(move(b), m);
            auto cand = ep.solve(k - 2, true);
            int heavy = min(m, mx + k - cand.used);
            cand.bag = ep.heap.addRun(cand.bag, heavy, 1);

            if (Engine::compareResults(ex, best, ep, cand) > 0)
                ans = ep.materialize(cand);
            else
                ans = ex.materialize(best);
        }
    }

    for (int i = 0; i < n; ++i) cout << ans[i] << " \n"[i + 1 == n];
}

```